CADT IDT

***W2** PRACTICE*

Native HTTP and Manual Routing

At the end of this practice, you can

- ✓ **Create** and run a native Node.js HTTP server
- ✓ **Manually implement** route handling using conditionals.

- ✓ Serve **static files** using fs.
- ✓ Parse form data from POST requests.
- ✓ Debug and enhance server code using console outputs.

Get ready before this practice!

- ✓ **Read** the following documents to understand Nodejs built-in HTTP module: https://nodejs.org/api/http.html

- ✓ **Read** the following documents to understand Anatomy of an HTTP Transaction: https://nodejs.org/en/learn/modules/anatomy-of-an-http-transaction

How to submit this practice?

- ✓ Once finished, push your **code to GITHUB**
- ✓ Join the **URL of your GITHUB** repository on LMS

*EXERCISE 1 – **ANALYZE***

Goal

- ✓ Identify and fix the bug.
- ✓ Understand the request-response cycle in Node.js using the http module.
- ✓ Explain the role of res.write() and res.end() in sending data back to the client.

For this exercise, you are provided with a minimal server.js file. Read and run the code and observe how it behaves.

```
// server.js
const http = require('http');

const server = http.createServer((req, res) => {
  res.write('Hello, World!');
  return res.endd();
});

server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Q1 – What error message do you see in the terminal when you access http://localhost:3000? What line of code causes it?

You will see an error like **"TypeError: res.endd is not a function"** in the terminal. This happens because of a typo in the line return res.endd(); - the correct method is res.end().

Q2 – What is the purpose of res.write() and how is it different from res.end()?

res.write() is used to send chunks of data to the client, while res.end() finishes the response and tells the server that all data has been sent.

Q3 – What do you think will happen if res.end() is not called at all?

If res.end() is never called, the request will hang and the browser will keep waiting because the server never signals that the response is finished.

Q4 – Why do we use http.createServer() instead of just calling a function directly?

We use http.createServer() because it sets up a server that listens for incoming HTTP requests and handles them properly.

Q5 – How can the server be made more resilient to such errors during development?

The server can be improved by using error handling like try-catch, adding proper logging, and using tools

*EXERCISE 2 – **MANIPULATE***

Goal

- ✓ Practice using req.url and req.method.
- ✓ Understand how manual routing mimics what frameworks (like Express) automate.
- ✓ Serve both plain text and raw HTML manually.

- For this exercise you will start with a **START CODE (EX-2)**

TASK 1 - Update the code above to add custom responses for these routes:

ROUTE	HTTP METHOD	RESPONSE
/about	**GET**	About us: at CADT, we love node.js!
/contact-us	**GET**	You can reach us vai email…
/products	**GET**	Buy one get one…
/projects	**GET**	Here are our awesome projects

Use VS Code's Thunder Client (or other tools (POSTMAN, INSOMIA) of your choice or curl on your terminal to make request.

Example output

curl http://localhost:3000/about ----------> About us: at CADT, we love node.js!

curl http://localhost:3000/contact-us ----------> You can reach us vai email…

TASK 2 – As we can see the complexitiy grow as we add more routes. Use switch statement to arrange the code into more organized structure.

❓ Reflective Questions

1. What happens when you visit a URL that doesn't match any of the three defined?

   When a user visits a URL that does not match any of the defined routes, the server runs the default case in the switch statement. In this case, the server sends back a **404 response**, which means "Not Found." It sets the response header to plain text and returns the message *"404 Not Found"*. This tells the browser that the requested page does not exist on the server.

2. Why do we check both the req.url and req.method?

   We check both req.url and req.method to make sure we are handling the correct request properly. The req.url tells us which path the client is requesting, while req.method tells us what type of action the client wants to perform (such as GET or POST). By checking both, we ensure that the server responds correctly based on both the requested route and the action.

3. What MIME type (Content-Type) do you set when returning HTML instead of plain text?

   When returning HTML, we set the Content-Type to **text/html**. This tells the browser to render the response as a web page instead of showing it as plain text.

4. How might this routing logic become harder to manage as routes grow?

   It becomes harder to read, debug, and maintain
   Adding new routes can lead to duplicate logic or errors
   It's difficult to organize routes by feature (e.g., users, products)

5. What benefits might a framework offer to simplify this logic?

   Frameworks make backend **cleaner, faster to build, and easier to manage**.

*EXERCISE 3 – **CREATE***

Goal

- ✓ Practice handling POST requests.
- ✓ Parse URL-encoded form data manually.
- ✓ Write and append to local files using Node.js' fs module.
- ✓ Handle async operations and errors gracefully.

- For this exercise you will start with a **START CODE EX-3**

TASK 1 - Extend your Node.js HTTP server to handle a **POST request** submitted from the contact form. When a user submits their name, the server should:

1. **Capture the form data** (from the request body).
2. **Log it to the console**.
3. **Write it to a local file** named submissions.txt**.**

Testing, go to /contact on broswer and test

Requirements

- Handle POST /contact requests.
- Parse raw application/x-www-form-urlencoded data from the request body.
- Write the name to a new line in submissions.txt.
- Send a success response to the client (HTML or plain text).